

## Chapter 46: String Pattern Matching

*Personal Notes*

---

"Does needle appear anywhere in haystack?" It sounds trivial — Java's `indexOf()` does it in one line. But under the hood, that operation has a fascinating story. The naive approach is  $O(n \times m)$ . KMP does it in  $O(n + m)$  with a clever pre-computed helper. Rabin-Karp uses a rolling hash to find matches in the SAME time but with a completely different technique that generalizes to multi-pattern search.

String pattern matching is the last major algorithm family in classical DSA. Beyond passing interviews, these ideas power `grep`, editor find-and-replace, DNA sequence matching, and plagiarism detection. This chapter covers KMP (deterministic, always  $O(n + m)$ ), Rabin-Karp (probabilistic, extends naturally to multi-pattern), and touches the Z-algorithm as a cleaner KMP alternative.

### 1. The Problem

Given a TEXT (haystack) of length  $n$  and a PATTERN (needle) of length  $m$ , find all positions where the pattern occurs in the text — or find just the first occurrence, or count occurrences. Three closely related variants.

```
text:      "a b x a b c a b c a b y"
pattern:   "a b c a b y"

Position 0: match "abxab..." vs pattern → mismatch at position 2
Position 1: match "bxab..." vs pattern → mismatch at position 0
...
Position 6: match "abcaby" vs pattern → FULL MATCH ✓

Return: position 6, or return true, or add 6 to a list of matches.
```

#### Real-world uses

- `grep` / editor find-and-replace
- Bioinformatics — searching DNA sequences
- Spam / virus signature detection
- Plagiarism detection (extended to multi-pattern)
- Web search — matching queries in indexed pages

### 2. Naive Approach — And Why It Fails

The obvious solution: check every possible starting position, and at each one, compare the pattern character-by-character.

```
public int naiveSearch(String text, String pattern) {
```

```

int n = text.length(), m = pattern.length();

for (int i = 0; i <= n - m; i++) {
    int j = 0;
    while (j < m && text.charAt(i + j) == pattern.charAt(j)) j++;
    if (j == m) return i;    // full match
}
return -1;
}

// Time: O(n × m) worst case
// Space: O(1)

```

### The Pathological Case

```

text:      "aaaaaaaaaab"
pattern:   "aaaab"

```

At each of the 6 possible start positions, we compare 4 or 5 characters before failing at the last one. Total:  $\sim 6 \times 5 = 30$  comparisons for  $n=11$ .

For  $n = 10^6$  and  $m = 10^3$ , that's  $10^9$  operations – way too slow.

**The wasted work:** When the naive algorithm mismatches at position  $i+j$ , it throws away ALL the information it just gathered. It jumps back to position  $i+1$  and starts over, potentially re-comparing characters that it already knew matched. KMP and Rabin-Karp both fix this — but in very different ways.

## 3. KMP — The Key Idea

KMP (Knuth-Morris-Pratt) avoids the redundant comparisons by PRE-PROCESSING the pattern. Specifically, it computes a helper array — the LPS (Longest Prefix that is also a Suffix) array — that tells you where to "jump back to" in the pattern after a mismatch, WITHOUT rewinding the text pointer.

After matching some characters and then failing, KMP asks:

"What's the LONGEST proper prefix of what I just matched that is also a SUFFIX of what I just matched?"

Whatever length that is, we know the pattern up to that length ALREADY matches the tail of the text – no need to re-compare.

Example – pattern "abcbabd", we matched "abcbab" and mismatched. "ab" is both a prefix and suffix of "abcbab". So we can safely assume "ab" matches the last 2 characters we compared –

resume comparison from pattern index 2 without moving the text pointer.

## 4. Building the LPS Array

For a pattern of length  $m$ , LPS is an array of length  $m$  where  $LPS[i]$  = length of the longest proper prefix of  $pattern[0..i]$  that is also a suffix. Built in  $O(m)$  using a self-similarity scan of the pattern.

### Example

```
pattern:  a b a b c a b c a b a b c
index:    0 1 2 3 4 5 6 7 8 9 10 11 12
LPS:      0 0 1 2 0 1 2 3 4 5 6 7 8
```

$LPS[2] = 1$ : "aba" – "a" is both prefix and suffix

$LPS[3] = 2$ : "abab" – "ab" is both prefix and suffix

$LPS[7] = 3$ : "abcbabca" – wait, actually  $LPS[7]$  should measure "abab c  
abc"

longest prefix that is also suffix

(The values depend on which prefix/suffix matches – computed incrementally in the code below.)

### Building Code

```
public int[] buildLPS(String pattern) {
    int m = pattern.length();
    int[] lps = new int[m];
    int len = 0;    // length of previous longest prefix-suffix

    for (int i = 1; i < m; ) {
        if (pattern.charAt(i) == pattern.charAt(len)) {
            len++;
            lps[i] = len;
            i++;
        } else {
            if (len != 0) len = lps[len - 1];    // fall back
            else { lps[i] = 0; i++; }
        }
    }
    return lps;
}

// Time:  $O(m)$ 
```

**The magic move —  $len = lps[len - 1]$ :** When characters mismatch, we don't reset  $len$  to 0 — we fall back to the previous LPS value, which represents the next-longest prefix-suffix match we can extend. This bit of self-recursion is what keeps LPS construction  $O(m)$  instead of  $O(m^2)$ .

## 5. KMP Search — Using the LPS Array

```
public List<Integer> kmpSearch(String text, String pattern) {
    List<Integer> matches = new ArrayList<>();
    int n = text.length(), m = pattern.length();
    if (m == 0) return matches;

    int[] lps = buildLPS(pattern);
    int i = 0, j = 0;    // i for text, j for pattern

    while (i < n) {
        if (text.charAt(i) == pattern.charAt(j)) {
            i++;
            j++;
            if (j == m) {
                matches.add(i - m);
                j = lps[j - 1];    // look for next match
            }
        } else {
            if (j != 0) j = lps[j - 1];    // fall back without moving i
            else i++;
        }
    }
    return matches;
}

// Time: O(n + m) – text pointer never moves backward
// Space: O(m)
```

### Why It's $O(n + m)$

The text pointer  $i$  only ever INCREASES. The pattern pointer  $j$  can bounce, but each character it processes is either a match ( $i$  advances) or a fallback ( $j$  strictly decreases). Amortized, the total work is  $O(n)$  for the search plus  $O(m)$  for building LPS. Beautiful.

```
text:      "a b a b c a b a b a b c a b c"
pattern:   "a b a b c a b"
```

After some comparisons at position 0, we match "ababcab" then mismatch.  $LPS[7-1] = LPS[6]$  tells us how far the pattern "already matches" the tail of what we compared – so we skip ahead in the pattern instead of restarting.

Result: found at index 8.

## 6. Rabin-Karp — The Rolling Hash Approach

Rabin-Karp takes a completely different route: HASH the pattern, then HASH each length- $m$  substring of the text and compare hashes. Two hashes matching is strong evidence of a real match; a mismatch is instant.

The key insight is the ROLLING HASH — the hash of  $\text{text}[1..m]$  can be computed from the hash of  $\text{text}[0..m-1]$  in  $O(1)$ , by subtracting the leading character's contribution and adding the trailing one.

### The Rolling Hash Formula

Treat characters as digits in base  $B$  (e.g.,  $B = 256$  or a prime).

$$\text{hash}(\text{"abc"}) = 'a' * B^2 + 'b' * B + 'c'$$

Given  $\text{hash}(\text{text}[i..i+m-1])$ , compute  $\text{hash}(\text{text}[i+1..i+m])$  as:

$$\text{new\_hash} = (\text{old\_hash} - \text{text}[i] * B^{(m-1)}) * B + \text{text}[i+m]$$

Then MOD by a large prime to keep the number manageable.

Cost per step:  $O(1) \rightarrow$  total  $O(n)$  for all windows.

### Code

```
public int rabinKarp(String text, String pattern) {
    int n = text.length(), m = pattern.length();
    if (m > n) return -1;

    int MOD = 1_000_000_007;
    int BASE = 256;

    // Precompute BASE^(m-1) mod MOD
    long h = 1;
    for (int i = 0; i < m - 1; i++) h = (h * BASE) % MOD;

    long patternHash = 0, windowHash = 0;
    for (int i = 0; i < m; i++) {
        patternHash = (patternHash * BASE + pattern.charAt(i)) % MOD;
        windowHash = (windowHash * BASE + text.charAt(i)) % MOD;
    }

    for (int i = 0; i <= n - m; i++) {
        if (patternHash == windowHash) {
            // Verify — hash collision protection
            boolean match = true;
            for (int j = 0; j < m; j++) {
                if (text.charAt(i + j) != pattern.charAt(j)) {
                    match = false;
                }
            }
        }
    }
}
```

```

        break;
    }
}
if (match) return i;
}

// Roll the hash to the next window
if (i < n - m) {
    windowHash = ((windowHash - text.charAt(i) * h) * BASE
        + text.charAt(i + m)) % MOD;
    if (windowHash < 0) windowHash += MOD;
}
}

return -1;
}

// Expected time: O(n + m). Worst case (many collisions): O(n × m).

```

**Hash collisions are possible:** Two different strings CAN have the same hash — that's why we always VERIFY a match by character comparison. A good hash function makes collisions rare; a bad one can degrade Rabin-Karp to  $O(n \times m)$ . Use a large prime modulus and, for adversarial inputs, consider double hashing (two independent hashes).

## 7. KMP vs Rabin-Karp

| Aspect               | KMP                              | Rabin-Karp                                      |
|----------------------|----------------------------------|-------------------------------------------------|
| Time (avg)           | $O(n + m)$ guaranteed            | $O(n + m)$ expected                             |
| Time (worst)         | $O(n + m)$ — deterministic       | $O(n \times m)$ — many hash collisions          |
| Preprocessing        | $O(m)$ for LPS array             | $O(m)$ for pattern hash + $\text{BASE}^{(m-1)}$ |
| Space                | $O(m)$                           | $O(1)$                                          |
| Multi-pattern search | Needs Aho-Corasick — complex     | Easy — hash multiple patterns, one text scan    |
| Implementation       | Trickier — LPS logic is subtle   | Simpler — rolling hash arithmetic               |
| Best for             | Single pattern, guaranteed speed | Multi-pattern or lightweight impl               |

**One-line rule of thumb:** For SINGLE pattern with guaranteed worst-case performance → KMP. For MULTIPLE patterns simultaneously, or when you just want simpler code → Rabin-Karp. In interviews, KMP is more prestigious but Rabin-Karp is often easier to code correctly under pressure.

## 8. The Z-Algorithm — A Cleaner Alternative

The Z-algorithm computes an array Z where  $Z[i]$  = length of the longest substring starting at position  $i$  that MATCHES A PREFIX of the string. Elegant, easy to code, and can be used for pattern matching by processing the string "pattern + '\$' + text".

```
public int[] zArray(String s) {
    int n = s.length();
    int[] z = new int[n];
    int l = 0, r = 0;

    for (int i = 1; i < n; i++) {
        if (i < r) z[i] = Math.min(r - i, z[i - l]);
        while (i + z[i] < n && s.charAt(z[i]) == s.charAt(i + z[i])) z[i]++;
        if (i + z[i] > r) { l = i; r = i + z[i]; }
    }
    return z;
}

// Time: O(n)

// For pattern matching, compute Z of "pattern + '$' + text".
// Any Z[i] == pattern.length() indicates a match at position (i -
// pattern.length() - 1) in text.
```

Z-algorithm is often used instead of KMP in competitive programming because the code is shorter and the concept is more intuitive. For interviews, KMP is more classical — but knowing Z is a nice bonus.

## 9. Classic Problem 1 — Implement strStr()

LeetCode 28. Given text and pattern, return the index of the first occurrence — or -1 if not found. Equivalent to Java's indexOf(). The go-to problem to demonstrate KMP or Rabin-Karp.

```
public int strStr(String haystack, String needle) {
    if (needle.isEmpty()) return 0;

    int[] lps = buildLPS(needle);
    int i = 0, j = 0;
    int n = haystack.length(), m = needle.length();

    while (i < n) {
        if (haystack.charAt(i) == needle.charAt(j)) {
            i++;
            j++;
            if (j == m) return i - m;
        } else {
            if (j != 0) j = lps[j - 1];
            else i++;
        }
    }
    return -1;
}
```

```

    }
}
return -1;
}

```

Naive approach also passes this problem — but interviewers often ask for  $O(n + m)$ , which is the KMP hint.

## 10. Classic Problem 2 — Repeated Substring Pattern

LeetCode 459. Given a string, check if it can be constructed by taking a substring and repeating it multiple times. Elegant KMP trick — no explicit search needed.

### The KMP Insight

Build the LPS array. If  $LPS[n-1] > 0$  AND  $n \% (n - LPS[n-1]) == 0$ , then the string is composed of repeating substrings of length  $(n - LPS[n-1])$ .

```

public boolean repeatedSubstringPattern(String s) {
    int n = s.length();
    int[] lps = buildLPS(s);
    int len = lps[n - 1];
    return len > 0 && n % (n - len) == 0;
}

// Example: "abab" → LPS = [0, 0, 1, 2] → len=2, n-len=2, 4 % 2 == 0 → TRUE
// Example: "aba" → LPS = [0, 0, 1] → len=1, n-len=2, 3 % 2 != 0 → FALSE

```

**Why this works:** If a string is "XXX" (X repeated 3 times), then the longest prefix-suffix (LPS) equals "XX". So  $n - LPS[n-1]$  gives the block length. If  $n$  is divisible by that block length, the whole string decomposes cleanly into repetitions.

## 11. Classic Problem 3 — Shortest Palindrome

LeetCode 214. Given a string, find the shortest palindrome you can form by adding characters to the FRONT. Uses KMP LPS on a cleverly constructed string.

### The Trick

Compute LPS on  $s + \# + \text{reverse}(s)$ .  $LPS[\text{last}]$  gives the length of the longest prefix of  $s$  that is also a palindrome. Add the remaining characters (reversed) to the front.

```

public String shortestPalindrome(String s) {
    String reversed = new StringBuilder(s).reverse().toString();
    String combined = s + "#" + reversed;
    int[] lps = buildLPS(combined);
    int palPrefixLen = lps[combined.length() - 1];
}

```



```

        // Characters we need to prepend: those NOT already in the palindromic
        prefix
        return reversed.substring(0, s.length() - palPrefixLen) + s;
    }

```

## 12. Classic Problem 4 — Longest Duplicate Substring

LeetCode 1044. Find the longest substring that appears at least TWICE in a string. Rabin-Karp shines here — combined with BINARY SEARCH on the length of the answer.

### The Strategy

- Binary-search the length  $L$  of the longest duplicate
- For each  $L$ , use rolling hash to find any repeated substring of that length
- If found, try LONGER (increase  $L$ ); otherwise SHORTER (decrease  $L$ )

```

public String longestDupSubstring(String s) {
    int lo = 1, hi = s.length() - 1;
    String best = "";

    while (lo <= hi) {
        int mid = (lo + hi) / 2;
        String found = findDup(s, mid);
        if (found != null) {
            best = found;
            lo = mid + 1;
        } else {
            hi = mid - 1;
        }
    }
    return best;
}

private String findDup(String s, int len) {
    long MOD = (1L << 32) - 1;
    long BASE = 26;

    long hash = 0, power = 1;
    for (int i = 0; i < len; i++) {
        hash = (hash * BASE + s.charAt(i)) % MOD;
        if (i > 0) power = (power * BASE) % MOD;
    }

    Map<Long, List<Integer>> seen = new HashMap<>();
    seen.computeIfAbsent(hash, k -> new ArrayList<>()).add(0);

    for (int i = 1; i <= s.length() - len; i++) {

```

```

        hash = ((hash - s.charAt(i - 1) * power) * BASE + s.charAt(i + len
- 1)) % MOD;
        if (hash < 0) hash += MOD;

        if (seen.containsKey(hash)) {
            String candidate = s.substring(i, i + len);
            for (int j : seen.get(hash)) {
                if (s.substring(j, j + len).equals(candidate)) return
candidate;
            }
        }
        seen.computeIfAbsent(hash, k -> new ArrayList<>()).add(i);
    }
    return null;
}

// Time: O(n log n) expected

```

This is a hard problem, but it beautifully combines rolling hash + binary search — a common pattern in advanced string problems.

## 13. When to Reach for Which Algorithm

| If you need to...                       | Use                                              |
|-----------------------------------------|--------------------------------------------------|
| Single pattern search (guaranteed fast) | KMP                                              |
| Multi-pattern search (many needles)     | Rabin-Karp with multiple hashes, or Aho-Corasick |
| Fast prototype / one-off tool           | Java's <code>indexOf()</code> — trust the JDK    |
| Check if string is periodic             | KMP LPS trick (Section 10)                       |
| Longest palindromic prefix              | KMP on <code>s + '#' + reverse(s)</code>         |
| Find longest duplicate substring        | Rabin-Karp + binary search                       |
| Anagram / permutation detection         | Sliding window + freq array (Ch 35)              |
| Fuzzy matching / edit distance          | DP — not KMP/RK (Ch 32)                          |
| Prefix / suffix operations              | Trie (Ch 36) — different tool                    |
| Regex-style pattern matching            | Java's <code>Pattern/Matcher</code> , or DFA/NFA |

## 14. Complexity Summary

| Algorithm                             | Time                    | Space      |
|---------------------------------------|-------------------------|------------|
| Naive matching                        | $O(n \times m)$         | $O(1)$     |
| KMP — Preprocess (LPS)                | $O(m)$                  | $O(m)$     |
| KMP — Search                          | $O(n)$                  | $O(m)$     |
| Rabin-Karp — Avg case                 | $O(n + m)$              | $O(1)$     |
| Rabin-Karp — Worst case               | $O(n \times m)$         | $O(1)$     |
| Z-Algorithm                           | $O(n + m)$              | $O(n + m)$ |
| Java <code>indexOf()</code> (default) | Usually $O(n \times m)$ | $O(1)$     |

**Yes — Java's `indexOf()` is  $O(n \times m)$ :** Despite being battle-tested, `String.indexOf()` uses the naive algorithm. For most practical strings it's fast because pathological cases are rare — but if a problem asks for guaranteed  $O(n + m)$ , you need to write KMP yourself. The interviewer knows this.

## 15. Common Mistakes

### Mistake 1: LPS off-by-one confusion

```
// After a mismatch — where do we fall back?
if (j != 0) j = lps[j - 1];      // ✓ correct — use PREVIOUS index
if (j != 0) j = lps[j];        // ✗ wrong — points past current position
```

### Mistake 2: Forgetting to advance i in the else branch

```
else {
    if (j != 0) j = lps[j - 1];
    else i++;      // MUST advance i, otherwise infinite loop
}
```

### Mistake 3: Not verifying hash matches in Rabin-Karp

Hash equality does NOT guarantee string equality — collisions exist. Always verify with a character-by-character comparison after a hash match. Skipping verification silently returns wrong results on colliding inputs.

### Mistake 4: Negative modulo in rolling hash

```
// In Java, negative numbers can appear after modulo:
if (hash < 0) hash += MOD;
```

```
// Forgetting this line silently breaks the hash for many test cases.
```

### Mistake 5: Wrong sentinel in KMP-based tricks

For "shortest palindrome" and similar problems, you concatenate strings with a separator ('#' or '\$') to prevent artificial matches spanning the boundary. Forgetting the separator makes LPS return wrong values.

### Mistake 6: Integer overflow in hash arithmetic

``char × BASE^m`` can overflow `int` quickly. Use `long` throughout, and `mod` aggressively. If you're getting weird collisions, this is often the reason.

### Mistake 7: Using Java's `indexOf` and claiming it's KMP

Some interviewers will directly ask you to implement string matching in  $O(n + m)$ . `indexOf()` is  $O(n \times m)$  — you must write your own KMP for this. Read the question carefully.

## 16. Best Practices

- For simple existence checks, use Java's `indexOf()` — it's fast enough in practice
- For guaranteed  $O(n + m)$ , implement KMP — memorize the LPS logic once
- For multi-pattern search, Rabin-Karp is often simpler than Aho-Corasick
- Always VERIFY hash matches in Rabin-Karp — never skip the character comparison
- Use long arithmetic and mod by a large prime for rolling hash
- Handle negative results from Java's modulo: `if (hash < 0) hash += MOD`
- For repeated-substring tricks, remember the LPS-based test: `n % (n - LPS[n-1])`
- Draw a small example (pattern of length 4-5) on paper before coding KMP

## 17. Quick Reference

| Concept          | Meaning                                                   |
|------------------|-----------------------------------------------------------|
| Pattern matching | Find needle in haystack in $O(n + m)$                     |
| KMP              | Uses LPS to avoid redundant comparisons                   |
| LPS array        | Longest Prefix that is also a Suffix                      |
| LPS fallback     | <code>j = lps[j - 1]</code> after mismatch — the key move |
| Rabin-Karp       | Rolling hash + verify on hash match                       |
| Rolling hash     | $O(1)$ hash update as window slides                       |
| Hash collision   | Same hash, different strings — MUST verify                |

| Concept                     | Meaning                                                  |
|-----------------------------|----------------------------------------------------------|
| <b>Z-algorithm</b>          | Alt to KMP — $Z[i]$ = LCP length between $s$ and $s[i:]$ |
| <b>Naive matching</b>       | $O(n \times m)$ — throws away information on mismatch    |
| <b>Multi-pattern search</b> | Rabin-Karp with a set of pattern hashes                  |

## 18. Practice Problems

Practice in order — foundational first, then LPS tricks, then rolling hash applications.

### Foundational

1. Implement `strStr()` (LeetCode 28) — using KMP or Rabin-Karp.
2. Find All Anagrams in a String (LeetCode 438) — sliding window, but Rabin-Karp variant works too.
3. Repeated Substring Pattern (LeetCode 459) — the KMP LPS trick.
4. Find the Index of the First Occurrence in a String (LeetCode 28) — standard KMP.

### LPS-Based Tricks

5. Shortest Palindrome (LeetCode 214) — LPS on  $s + \# + \text{reverse}(s)$ .
6. Longest Prefix Which Is Also Suffix — the  $\text{LPS}[n-1]$  value.
7. Count occurrences of a pattern in a text.
8. Longest Happy Prefix (LeetCode 1392) — LPS directly.

### Rolling Hash Applications

9. Longest Duplicate Substring (LeetCode 1044).
10. Repeated DNA Sequences (LeetCode 187).
11. Rabin-Karp implementation from scratch.
12. Distinct Echo Substrings (LeetCode 1316).

### Advanced

13. Sum of Scores of Built Strings (LeetCode 2223) — Z-algorithm.
14. String Matching in an Array (LeetCode 1408).
15. Longest Common Substring — using DP or suffix arrays.
16. Palindrome Partitioning IV (LeetCode 1745) — combines multiple string techniques.

## 19. Final Takeaway

String pattern matching feels like a solved problem — Java has `indexOf()`, after all. But under the hood, KMP and Rabin-Karp represent two beautiful different approaches: DETERMINISTIC preprocessing (KMP) versus PROBABILISTIC rolling hash (Rabin-Karp). Both are elegant, and both come up in Google interviews often enough to be worth mastering.

For interviews, learn KMP thoroughly — the LPS array is the most-tested concept in this space. Rabin-Karp is a strong bonus, especially for multi-pattern search or when the interviewer allows it. Z-algorithm is a nice-to-have. The classic problems (Implement `strStr`, Repeated Substring Pattern, Shortest Palindrome, Longest Duplicate Substring) exercise every important variant of the pattern.

**Key Takeaway:** Naive matching is  $O(n \times m)$ . KMP achieves  $O(n + m)$  using an LPS array that tells you where to fall back after a mismatch — text pointer never rewinds. Rabin-Karp uses a rolling hash: hash the pattern, hash each window, verify on hash match. KMP is deterministic and best for single patterns; Rabin-Karp is probabilistic and generalizes to multi-pattern. Master the LPS fallback ( $j = \text{lps}[j-1]$ ) and rolling hash update — those two ideas unlock the entire chapter.

## 20. What's Next?

With string matching covered, the classical DSA toolkit is essentially COMPLETE. Remaining specialized topics:

- Number Theory basics — modular arithmetic, primes, GCD tricks, combinatorics
- Randomized Algorithms — Quickselect, Reservoir Sampling, Monte Carlo
- Suffix Arrays / Automata — advanced string data structures
- Max Flow / Min Cut — Ford-Fulkerson, Edmonds-Karp
- Computational Geometry — convex hull, line sweep, closest pair
- Advanced DP — knapsack variants, digit DP, matrix exponentiation
- System Design fundamentals — the natural big step for senior interviews